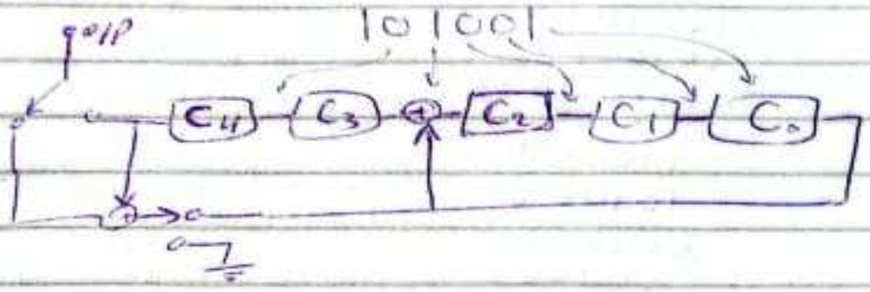
Assignment 1

data: 0110100110101011101

generator: 101001 (6) → 5 bits

5 4 3 2 1 0

1. Diagram



2.

```
             01110111110111000
101001 ) 011010011010101110100000
          110100
          101001
         x111011
           101001
         x100101
           101001
          x011000
            110001
            101001
           x110000
             101001
           x110011
              101001
            x110100
               101001
              x111011
                 101001
                x100101
                  101001
                 x011001
                   110010
                   101001
                  x110111
                    101001
                   x111100
                     101001
                    x101010
                      101001
                     x000110
                          11000
```

CRC: 11000

3. No error

```
          0111011111101111000
101001 | 011010011010101110111000
         110100
         101001
        x111011
          101001
         x100101
          101001
         x011000
              110001
              101001
             x110000
               101001
              x110011
                101001
               x110100
                 101001
                x111011
                  101001
                 x100101
                   101001
                  x011001
                       110010
                       101001
                      x110111
                       101001
                      x111101
                       101001
                      x101001
                       101001
                       x00000
                         00000   as remainder.
```

4. Error in 6th bit

before: 01101⓪ invert.

after; data. : 0110110110101011101

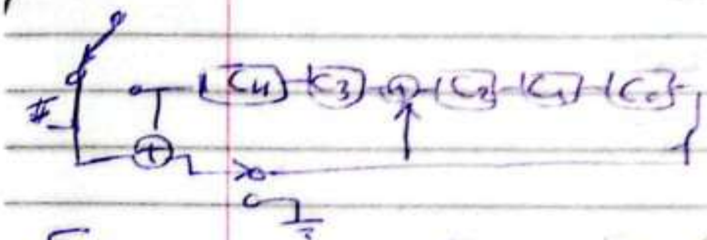
```
        011100101001100100 1
101001 ) 0110110110101011101 11000
         110110
         101001
         x111111
          101001
          x101101
           101001
           x001000
             100010
             101001
             x010111
               101110
               101001
               x001111
                 111111
                 101001
                 x101100
                  101001
                  x001011
                     101111
                     101001
                     x001100
                       110000
                       101001
                       [illegible]
                       x11001
```

1100 1

Remainder ≠ 0

& error

data: 011011011



5.

	C4	C3	C2	C1	C0	I⊕C4	I⊕C4⊕C2	I
		C4		C2	C1	C0	C3	
①	0	0	0	0	0	0	0	0
②	0	1	0	0	0	1	1	1
③	1	1	0	0	1	1	1	1
④	1	1	0	1	1	1	1	0
⑤	1	1	1	1	1	0	0	1
Sixth cycle → ⑥	1	0	1	1	0	0	1	1
	0	1	1	0	0	0	1	0

MATLAB code

```matlab
% Telecommunication Networks - Assignment 1 CRC Validation
clc; clear;

% 1. Setup Data and Polynomial
% Data stream given in the assignment
data = [0 1 1 0 1 0 0 1 1 0 1 0 1 0 1 1 1 0 1];

% Generator polynomial x^5 + x^3 + 1 translates to 101001
poly = [1 0 1 0 0 1];
k = length(poly) - 1; % Number of CRC bits (degree of polynomial is 5)

%% --- Part 2: Calculate CRC Field & Transmitted Stream ---
fprintf('--- Part 2: Transmitted Stream ---\n');

% Append k zeros to the original data
padded_data = [data, zeros(1, k)];
temp = padded_data;

% Perform Modulo-2 division (XOR) simulating long division
for i = 1:length(data)
    if temp(i) == 1
        % XOR the polynomial with the current bits
        temp(i:i+k) = bitxor(temp(i:i+k), poly);
    end
end

% The remainder is the last k bits
crc_field = temp(end-k+1:end);
transmitted_stream = [data, crc_field];

disp('1. Calculated CRC Field:');
disp(crc_field);
disp('2. Full Transmitted Stream:');
disp(transmitted_stream);

%% --- Part 3: Calculate Remainder at Receiver (No Error) ---
fprintf('\n--- Part 3: Receiver Check (No Error) ---\n');

temp_rx = transmitted_stream;

% Perform Modulo-2 division at the receiver
for i = 1:length(data)
    if temp_rx(i) == 1
        temp_rx(i:i+k) = bitxor(temp_rx(i:i+k), poly);
    end
end

remainder_no_error = temp_rx(end-k+1:end);
```

```
disp('Receiver Remainder (Should be all zeros):');
disp(remainder_no_error);

%% --- Part 4: Calculate Remainder if Error in 6th Bit ---
fprintf('\n--- Part 4: Receiver Check (Error in 6th bit) ---\n');

% Introduce error in the 6th transmitted bit
rx_with_error = transmitted_stream;
% The ~ operator flips the bit (0 becomes 1, or 1 becomes 0)
rx_with_error(6) = ~rx_with_error(6);

temp_rx_error = rx_with_error;

% Perform Modulo-2 division at the receiver on the corrupted data
for i = 1:length(data)
    if temp_rx_error(i) == 1
        temp_rx_error(i:i+k) = bitxor(temp_rx_error(i:i+k), poly);
    end
end

remainder_with_error = temp_rx_error(end-k+1:end);
disp('Receiver Remainder (Error detected):');
disp(remainder_with_error);
```

command window output

```
--- Part 2: Transmitted Stream ---

1. Calculated CRC Field:

   1   1   0   0   0



2. Full Transmitted Stream:

  Columns 1 through 19


   0   1   1   0   1   0   0   1   1   0   1   0   1   0   1   1   1   0   1


  Columns 20 through 24


   1   1   0   0   0
```

```
--- Part 3: Receiver Check (No Error) ---

Receiver Remainder (Should be all zeros):

     0     0     0     0     0



--- Part 4: Receiver Check (Error in 6th bit) ---

Receiver Remainder (Error detected):

     1     1     0     0     1


>>
```